선행 개념 정리: 딥러닝의 "왜?"를 따라가는 맥락 가이드

> 이 문서는 강의에 등장하는 모든 기법들이 **어떤 문제를 해결하려고 나왔는지**, "문제 → 첫 시도 → 한계 → 개선된 해결책"의 흐름으로 정리합니다. 수식을 외우기 전에 이 맥락을 먼저 읽으면, 수식이 "왜 이렇게 생겼는지"가 보입니다.

---

1. 뉴럴넷이 "학습"한다는 것의 의미

1.1 문제: 함수를 수동으로 설계할 수 없다

"이미지가 고양이인지 판별하는 프로그램"을 if-else로 짜는 것은 불가능하다. 귀의 위치, 색깔, 종류가 너무 다양하다.

1.2 해결: 데이터로부터 함수를 "찾게" 하자

뉴럴넷은 **파라미터화된 함수** $f_\theta(x)$이다. 데이터를 보여주면서 θ를 조정하여 원하는 함수에 가까워지게 한다.

1.3 어떻게 조정하는가? → 경사 하강법 (Gradient Descent)

1. 현재 파라미터 θ로 예측값 $\hat{y} = f_\theta(x)$를 계산
2. 정답 y와 비교하여 **손실(Loss)** $\mathcal{L}(\hat{y}, y)$를 계산
3. θ를 $\mathcal{L}$이 줄어드는 방향으로 조금 이동:

$$\theta \leftarrow \theta - \eta \cdot \frac{\partial \mathcal{L}}{\partial \theta}$$

비유: 눈을 감고 산에서 내려오기. 발밑의 경사(기울기)를 느끼고, 가장 가파르게 내려가는 방향으로 한 발 내딛는다.

1.4 왜 "기울기"가 필요한가? → 체인룰(Chain Rule)의 역할

$\frac{\partial \mathcal{L}}{\partial \theta}$를 구해야 하는데, 뉴럴넷은 함수의 합성이다:

$$\hat{y} = f_3(f_2(f_1(x; \theta_1); \theta_2); \theta_3)$$

θ_1을 바꾸면 f_1의 출력이 바뀌고, 그것이 f_2, f_3을 거쳐 최종 출력과 손실에 영향을 준다. 이 **인과적 연쇄**를 곱으로 표현한 것이 체인룰이다:

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{\partial \mathcal{L}}{\partial f_3} \cdot \frac{\partial f_3}{\partial f_2} \cdot \frac{\partial f_2}{\partial f_1} \cdot \frac{\partial f_1}{\partial \theta_1}$$

왜 곱셈인가? 비율의 비율의 비율이기 때문이다.

- θ_1이 1% 바뀌면 f_1 출력이 3% 바뀌고 (×3)
- f_1이 1% 바뀌면 f_2 출력이 2% 바뀌고 (×2)
- f_2가 1% 바뀌면 $\mathcal{L}$이 5% 바뀌면 (×5)
- 총 변화: 3 × 2 × 5 = 30% → **곱셈이 자연스럽다**

역전파(Backpropagation): 체인룰을 **뒤에서부터** 계산하는 알고리즘. 앞에서부터 하면 중복 계산이 생기지만, 뒤에서부터 하면 한 번씩만 계산하면 된다 (Dynamic Programming).

---

2. 활성 함수(Activation Function) — 왜 필요한가?

2.1 문제: 선형 변환의 합성은 선형이다

행렬곱의 연속: $f_2(f_1(x)) = W_2(W_1x) = (W_2W_1)x = W'x$

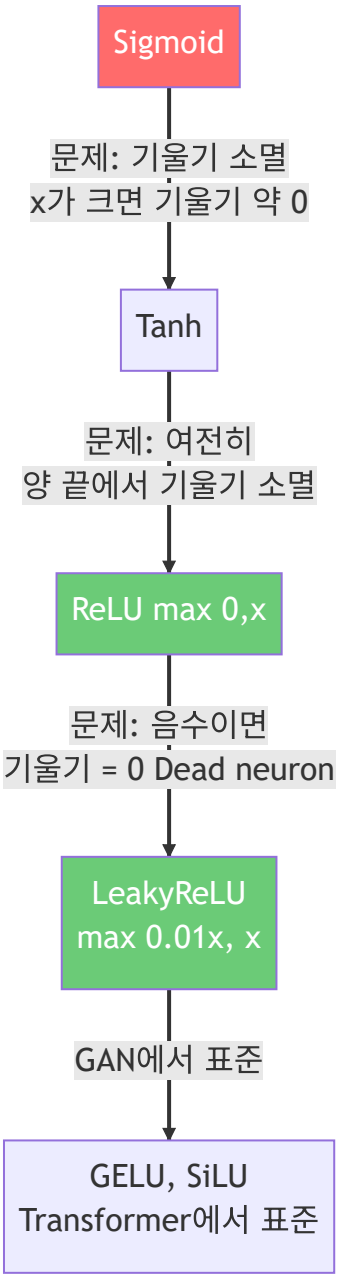
층을 아무리 깊게 쌓아도 **하나의 선형 변환**과 같다. XOR조차 학습 불가능.

2.2 해결: 비선형 함수를 중간에 끼운다

$$h = \sigma(Wx + b)$$

σ가 비선형이면, 층을 쌓을수록 점점 복잡한 함수를 표현할 수 있다 (Universal Approximation Theorem).

2.3 어떤 활성 함수를 쓰는가? — 진화의 역사



Sigmoid의 기울기 소멸 문제: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$. x가 크거나 작으면 $\sigma(x) \approx 0$ 또는 1이므로 $\sigma' \approx 0$. 체인룰에서 이런 작은 값이 여러 번 곱해지면 기울기가 0에 수렴한다.

ReLU가 혁명적인 이유: $x > 0$이면 기울기가 항상 1. 곱해도 줄어들지 않는다. 덕분에 깊은 네트워크가 가능해졌다.

GAN에서 LeakyReLU를 쓰는 이유: 판별자에서 $x < 0$인 영역도 약간의 기울기(0.2)를 흘려보내어, 학습이 완전히 멈추는 것을 방지한다.

---

3. 손실 함수(Loss Function) — "무엇을 최적화하는가"의 본질

3.1 왜 MSE(L2 Loss)를 쓰는가?

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_i (y_i - \hat{y}_i)^2$$

확률적 해석: 출력이 가우시안 분포를 따른다고 가정하면 ($p(y|x) = \mathcal{N}(\hat{y}, \sigma^2)$), **negative log-likelihood를 최소화하는 것이 MSE를 최소화하는 것과 동치**이다.

$$-\log p(y|x) = -\log \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\hat{y})^2}{2\sigma^2}\right) = \frac{(y-\hat{y})^2}{2\sigma^2} + \text{const}$$

σ가 상수이면, 이것을 최소화하는 것은 $(y-\hat{y})^2$를 최소화하는 것과 같다.

핵심: MSE를 쓰는 것은 "출력이 가우시안을 따른다"는 **가정**을 하는 것이다.

3.2 왜 Cross-Entropy를 쓰는가?

$$\mathcal{L}_{\text{CE}} = -\sum_k y_k \log \hat{y}_k$$

확률적 해석: 출력이 카테고리컬 분포를 따른다고 가정하면, negative log-likelihood가 cross-entropy이다.

- 분류 문제: 출력이 확률 분포 → Cross-Entropy
- 회귀 문제: 출력이 연속값 → MSE

픽셀 예측에서의 차이:

- MSE: 픽셀 값을 연속값으로 예측 → 평균적인 값으로 수렴 → **흐릿한 이미지**
- CE (256-way): 픽셀 값을 256개 카테고리 중 하나로 분류 → **선명한 이미지**

이것이 PixelRNN/CNN이 256-way softmax를 사용하는 이유다. 그리고 VAE 이미지가 흐릿한 이유도 근본적으로 MSE(가우시안 가정)에서 비롯된다.

3.3 Reconstruction Loss의 본질 — "복원"이 왜 확률과 연결되는가?

VAE의 Reconstruction Term:

$$\mathbb{E}_{q(z|x)}[\log p_\theta(x|z)]$$

이것이 "복원 품질"을 나타내는 이유:

$p_\theta(x|z)$는 "잠재 코드 z가 주어졌을 때 원본 x가 나올 확률"이다.

- **가우시안 디코더** 가정: $p(x|z) = \mathcal{N}(x; \mu_\theta(z), \sigma^2 I)$

$$\log p(x|z) \propto -\|x - \mu_\theta(z)\|^2$$

→ **MSE를 최소화 = 복원 확률을 최대화**

- **베르누이 디코더** 가정: 각 픽셀이 독립적 확률

$$\log p(x|z) = \sum x_k \log \hat{x}_k + (1 - x_k) \log(1 - \hat{x}_k)$$

→ **Cross-Entropy를 최소화 = 복원 확률을 최대화**

결론: 어떤 디코더를 가정하느냐에 따라 reconstruction loss의 형태가 달라지지만, 본질은 같다 — **"z에서 x를 잘 복원할 확률을 높여라."**

---

4. 기울기 소멸/폭발 — 딥러닝의 근본 난제

4.1 문제: 깊은 네트워크를 학습할 수 없다

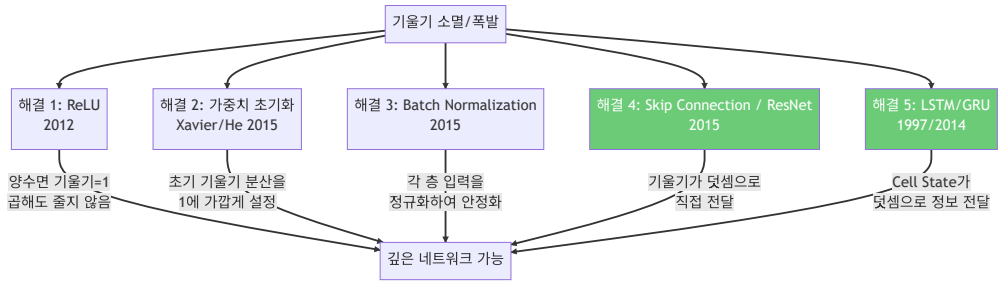
체인룰에서 기울기는 **각 층 기울기의 곱**이다:

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \prod_{l=1}^{L} \frac{\partial h_l}{\partial h_{l-1}} \cdot \frac{\partial \mathcal{L}}{\partial h_L}$$

- 각 항이 1보다 작으면: $0.9^{100} = 0.0000265$ → **소멸**
- 각 항이 1보다 크면: $1.1^{100} = 13781$ → **폭발**

10층만 넘어가도 학습이 불안정해진다.

4.2 해결책의 역사 — 각각이 어떤 문제를 풀었는가



4.3 Skip Connection (ResNet) — 가장 중요한 해결책

$$y = F(x) + x$$

왜 이것이 기울기 소멸을 막는가?

역전파 시:

$$\frac{\partial y}{\partial x} = \frac{\partial F(x)}{\partial x} + 1$$

"+1" 때문에 기울기가 아무리 작아져도 최소 1이 보장된다. 100층을 곱해도:

$$(? + 1) \times (? + 1) \times \cdots \times (? + 1) \geq 1^{100} = 1$$

이것이 1000층짜리 네트워크도 학습 가능하게 만든 핵심이다.

이 원리가 등장하는 모든 곳:

- **ResNet**: 이미지 분류
- **LSTM**: Cell State의 덧셈 ($C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$)
- **Transformer**: Residual connection ($x + \mathrm{Attention}(x)$)
- **U-Net**: Skip connection (인코더 → 디코더)
- **Diffusion**: 노이즈 추가/제거 과정 자체가 작은 residual 변환

모든 현대 딥러닝은 Skip Connection 위에 세워져 있다.

---

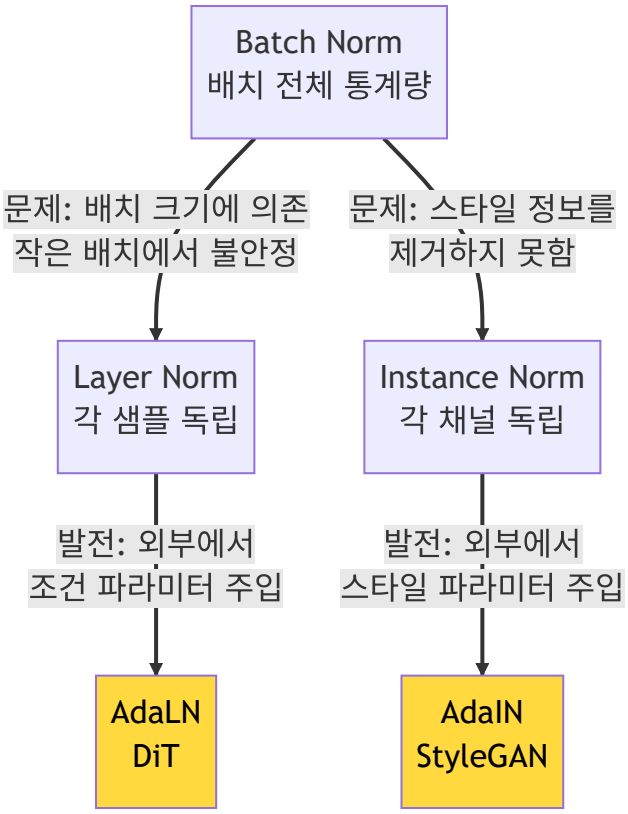
5. 정규화(Normalization) — 왜 필요한 것이 이렇게 많은가?

5.1 근본 문제: Internal Covariate Shift

깊은 네트워크에서 앞 레이어의 가중치가 바뀌면, 뒷 레이어의 **입력 분포가 바뀐다**. 뒷 레이어는 "움직이는 과녁"을 맞춰야 하는 셈이다.

5.2 해결의 진화

기법	정규화 범위	주요 사용처	핵심 효과
Batch Norm	배치 × 공간	분류, 일반 CNN	학습 안정화
Layer Norm	채널 × 공간	Transformer	배치 크기 무관
Instance Norm	공간만 (이미지 1장)	스타일 변환	**스타일 정보 제거**
Group Norm	채널 그룹 × 공간	소규모 배치	BN의 대안
AdaIN	IN + 외부 스타일 주입	StyleGAN	**스타일 주입**
AdaLN	LN + 외부 조건 주입	DiT	**시간/클래스 주입**



핵심 흐름: BN에서 시작해서, "정규화 후 복원하는 파라미터를 어디서 가져오는가?"에 따라 분화했다:

- **BN/LN/IN**: 학습 가능한 고정 파라미터 γ, β
- **AdaIN**: 스타일 코드 w에서 동적으로 생성
- **AdaLN**: 시간/클래스 임베딩에서 동적으로 생성

---

6. 컨볼루션(Convolution) — 왜 이미지에 CNN을 쓰는가?

6.1 문제: FC 레이어로 이미지를 다루면?

28×28 MNIST: 784개 입력 → FC → 관리 가능 256×256 컬러: 196,608개 입력 → FC → **수십억 파라미터** → 불가능

6.2 이미지의 핵심 성질 2가지

지역성(Locality): 어떤 픽셀의 의미는 그 **주변 픽셀**에 의해 결정된다. 이미지 왼쪽 구석의 픽셀이 오른쪽 구석에 직접 영향을 주는 경우는 드물다.

평행이동 불변성(Translation Invariance): 고양이 귀가 어디에 있든 "고양이 귀"다. 위치에 따라 다른 필터를 학습할 이유가 없다.

6.3 컨볼루션이 이것을 어떻게 반영하는가

- **지역성** → 작은 필터 (3×3, 5×5)로 주변만 봄 → 파라미터 수 급감
- **불변성** → 같은 필터를 이미지 전체에 **공유** → 어디서든 같은 패턴 감지

$$\text{출력}(i,j) = \sum_{m=-1}^{1} \sum_{n=-1}^{1} \text{입력}(i+m, j+n) \cdot \text{필터}(m,n)$$

이것은 "필터 패턴과 입력 영역의 유사도"를 측정하는 연산이다. 필터가 수평 엣지 패턴이면, 수평 엣지가 있는 곳에서 출력이 크다.

6.4 Receptive Field — 왜 깊은 CNN이 필요한가?

3×3 필터 하나는 3×3 영역만 본다. 두 층 쌓으면 5×5, 세 층 쌓으면 7×7... 층을 깊이 쌓을수록 더 넓은 영역의 정보를 통합할 수 있다.

깊은 CNN = 지역적 패턴(엣지) → 중간 패턴(코너, 질감) → 글로벌 패턴(물체 전체)의 계층적 표현

6.5 Stride와 Pooling — 왜 해상도를 줄이는가?

- **Stride 2 Conv**: 필터를 2칸씩 건너뛰며 적용 → 출력 해상도 1/2
- **Max Pooling**: 2×2 영역에서 최댓값만 선택 → 출력 해상도 1/2

왜 줄이는가?

1. 계산량 감소 (가로·세로 각 1/2 → 전체 1/4)
2. Receptive field 빠르게 확장
3. 점점 추상적인 표현 학습 (세부 → 전체)

DCGAN에서 Pooling을 안 쓰는 이유: Pooling은 정보를 버린다 (max만 취하고 나머지 버림). 생성 모델에서는 정보 보존이 중요하므로, **Strided Convolution**으로 대체한다 (학습 가능한 다운샘플링).

6.6 Transposed Convolution — 어떻게 해상도를 "올리는가"?

생성 모델의 핵심 연산: 저해상도 → 고해상도

일반 Conv는 큰 것을 작게 만든다. Transposed Conv는 작은 것을 크게 만든다.

작동 원리: 입력의 각 픽셀을 "씨앗"으로, 필터 크기만큼의 영역에 값을 "뿌린다". 겹치는 부분은 합산.

체커보드 아티팩트: Transposed Conv의 겹침이 불균일하면 격자 패턴이 나타난다. 이를 피하기 위해 **Upsample + Conv** (먼저 보간으로 키우고 일반 Conv 적용)를 쓰기도 한다.

---

7. RNN과 LSTM — 시퀀스 처리의 진화

7.1 문제: CNN은 고정 길이 입력만 받는다

"I love cats" (3단어)와 "The quick brown fox jumps" (5단어)를 어떻게 같은 네트워크로 처리하나?

7.2 해결: 한 번에 하나씩 처리 (RNN)

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

이전 상태 h_{t-1}과 현재 입력 x_t를 받아 새 상태를 만든다. 길이에 무관하게 동작한다.

7.3 문제: 긴 시퀀스에서 기억을 잃는다 (Vanishing Gradient)

h_{100}에서 h_1까지 역전파하면: W^{100} → 기울기 소멸/폭발

비유: 100명이 줄 서서 귓속말 전달 → 원래 메시지 왜곡

7.4 해결: LSTM — "메모장"을 추가

Cell State C_t: 정보가 거의 변형 없이 직선으로 흐르는 통로

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

이것이 **덧셈**이라는 것이 핵심이다. 기울기가 곱해지지 않고 더해지므로, 먼 과거의 정보도 유지될 수 있다.

3개의 게이트:

게이트	수식	비유
Forget f_t	$\sigma(W_f[h_{t-1}, x_t])$	"이 메모 아직 필요해?"
Input i_t	$\sigma(W_i[h_{t-1}, x_t])$	"이건 기록할 가치가 있어?"
Output o_t	$\sigma(W_o[h_{t-1}, x_t])$	"지금 뭘 말해야 해?"

모든 게이트가 σ (sigmoid, 0~1)를 사용하는 이유: "얼마나 열지/닫을지"를 연속적으로 제어하기 위해. 0이면 완전 차단, 1이면 완전 통과.

7.5 PixelRNN에서 왜 LSTM을 쓰는가?

이미지를 "왼쪽 위 → 오른쪽 아래" 순서로 읽으면 1D 시퀀스가 된다. 현재 픽셀을 예측하려면 이전 픽셀들의 **장기 의존성**이 필요하다 (예: 왼쪽 위의 하늘 색이 오른쪽 아래의 하늘 색에 영향). LSTM이 이 장기 의존성을 보존한다.

---

8. Attention과 Transformer — CNN/RNN을 넘어서

8.1 문제: CNN은 지역적, RNN은 순차적

- **CNN**: 3×3 필터는 주변만 봄. 먼 위치의 관계를 포착하려면 매우 깊어야 함.
- **RNN**: 한 스텝씩 순차 처리. 병렬화 불가. 긴 시퀀스에서 여전히 정보 손실.

8.2 해결: 모든 위치 쌍을 동시에 비교 (Self-Attention)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

각 단계의 의미:

1. QK^T: 모든 위치 쌍의 유사도 계산. N개 토큰이면 $N \times N$ 행렬.
    - "이 위치가 다른 모든 위치와 얼마나 관련 있는가?"
2. $/\sqrt{d_k}$: 스케일 조정.
    - 왜? 내적의 크기가 d_k에 비례하여 커짐. softmax가 saturate되는 것 방지.
3. **softmax**: 유사도를 확률(가중치)로 변환.
    - 가장 관련 높은 위치에 집중하게 만듦.
4. $\times V$: 관련 높은 위치의 정보를 가중 합산.
    - "관련 높은 곳의 정보를 모아서 내 표현을 업데이트"

비유: 시험 공부할 때, 교과서의 모든 문장을 보고 "이 문제와 관련 있는 문장"을 찾아 참고하는 것. CNN은 앞뒤 3문장만, RNN은 순서대로 읽으며 기억에 의존. Attention은 **관련 있는 문장을 직접 찾아간다**.

8.3 Multi-Head Attention — 왜 여러 개의 Attention을 동시에?

하나의 Attention은 하나의 "관점"으로만 관련성을 본다. 여러 Head는 **다양한 관점**에서 관련성을 본다:

- Head 1: 문법적 관계 (주어-동사)
- Head 2: 의미적 관계 (동의어)
- Head 3: 위치적 관계 (인접)

8.4 Cross-Attention — 조건부 생성의 핵심

Self-Attention: Q, K, V가 **같은 입력**에서 유도

Cross-Attention: Q는 **이미지 피처**, K와 V는 **텍스트 임베딩**에서 유도

이것이 "이미지의 각 위치가 텍스트의 어떤 단어를 참조할지"를 결정한다. Stable Diffusion에서 텍스트 조건을 주입하는 핵심 메커니즘.

8.5 Causal Mask — 왜 "미래를 못 보게" 하는가?

자기회귀 생성에서, 토큰 i를 생성할 때 $i+1, i+2, \ldots$는 아직 생성되지 않았다. 이들을 참조하면 **치팅**이다.

Causal Mask: Attention 행렬의 위쪽 삼각(미래 토큰)을 $-\infty$로 설정 → softmax 후 0이 됨 → 참조 불가

이것이 PixelCNN의 Masked Convolution과 동일한 역할이다. GPT, VQ-GAN Stage 2에서 사용.

---

9. 확률 분포와 생성 모델 — "분포를 학습한다"는 것의 의미

9.1 왜 확률을 다루는가?

생성 모델의 목표: "이 이미지들과 비슷한 새 이미지를 만들어라."

이것을 수학적으로 표현하면: "이 이미지들의 확률 분포 $p(x)$를 학습하고, 거기서 샘플링하라."

9.2 Maximum Likelihood Estimation (MLE) — 가장 기본적인 학습 원리

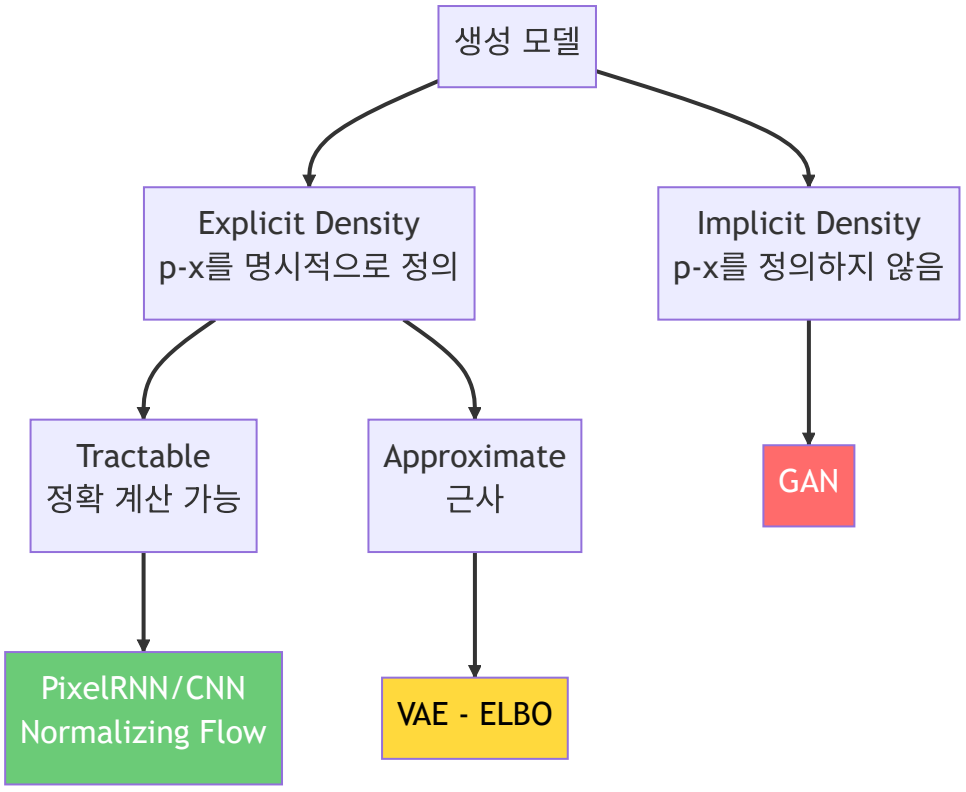
$$\max_{\theta} \prod_{i=1}^{N} p_\theta(x_i) \iff \max_{\theta} \sum_{i=1}^{N} \log p_\theta(x_i)$$

"훈련 데이터의 확률이 가장 높아지도록 파라미터를 조정하라."

로그를 취하는 이유:

1. 곱 → 합으로 변환 (수치적으로 안정)
2. 최적점은 동일 (로그는 단조증가)

9.3 Explicit vs Implicit Density



- **PixelRNN**: $p(x) = \prod p(x_i|x_{<i})$ → 정확한 likelihood
- **Flow**: 변수 변환으로 정확한 likelihood
- **VAE**: $p(x)$를 직접 못 구해서 ELBO(하한)를 최적화
- **GAN**: $p(x)$를 아예 정의하지 않고, "진짜처럼 보이게" 만들기만 함

9.4 KL Divergence와 ELBO의 관계 — 왜 "하한"을 최적화하는가?

$\log p(x)$를 직접 최적화하고 싶지만, $p(x) = \int p(x|z)p(z)dz$라는 적분이 intractable.

ELBO 유도의 핵심:

$$\log p(x) = \text{ELBO} + D_{KL}(q(z|x)\|p(z|x))$$

$D_{KL} \geq 0$이므로 $\log p(x) \geq \text{ELBO}$. ELBO를 올리면 $\log p(x)$도 올라간다 (하한을 올리면 본체도 올라감).

비유: 건물의 정확한 높이를 모르지만, 1층 높이(ELBO)는 알 수 있다. 1층을 높이면 건물 전체도 높아진다 (보장은 아니지만 대체로 맞다).

---

10. 잠재 변수(Latent Variable) — 왜 z를 도입하는가?

10.1 문제: 데이터가 너무 복잡하다

이미지 분포 $p(x)$를 직접 모델링하기엔 차원이 너무 높고 분포가 너무 복잡하다.

10.2 해결: 저차원 "원인"을 도입

이미지가 생성되는 과정을 상상하면:

1. "원인" z (나이, 성별, 포즈, 조명...)가 먼저 결정
2. z에 따라 이미지 x가 생성

$$p(x) = \int p(x|z)p(z)dz$$

비유: 얼굴 사진을 만들 때, 먼저 "30대 여성, 웃는 표정, 정면"이라는 잠재 요인을 결정하고, 그에 맞는 사진을 생성한다.

10.3 각 모델에서 z의 역할

모델	z의 형태	z의 역할
VAE	연속 벡터 $z \in \mathbb{R}^d$	데이터의 연속적 표현
VQ-VAE	이산 인덱스 $z \in \{1, ..., K\}^{h \times w}$	코드북의 인덱스
GAN	랜덤 노이즈 $z \sim \mathcal{N}(0, I)$	생성의 "씨앗"
Diffusion	$x_T \sim \mathcal{N}(0, I)$ (순수 노이즈)	시작점
Flow	$z \sim \mathcal{N}(0, I)$	가역 변환의 입력

---

11. 확률적 연산과 역전파 — Reparameterization의 필요성

11.1 문제: "랜덤"에서 기울기를 구할 수 없다

VAE에서 $z \sim \mathcal{N}(\mu, \sigma^2)$를 샘플링해야 한다. 그런데:

역전파의 원리: 출력의 변화가 입력의 변화에 어떻게 의존하는지 추적.

"μ를 0.01 올리면 z가 얼마나 변하는가?" → z가 확률적이라 답할 수 없다. 같은 μ에서도 매번 다른 z가 나온다.

11.2 해결: 확률적 요소를 외부로 분리

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

이제 μ를 0.01 올리면 z도 정확히 0.01 올라간다. ϵ은 외부에서 주입된 "상수 같은 노이즈"이므로 μ, σ에 대한 기울기가 존재한다.

이 트릭이 등장하는 모든 곳:

- **VAE**: $z = \mu + \sigma \odot \epsilon$
- **Diffusion**: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ (같은 원리)
- **Gumbel-Softmax**: 이산 분포에서의 reparameterization

---

12. Stop Gradient (sg) — 왜 기울기를 일부러 차단하는가?

12.1 VQ-VAE에서의 핵심 문제

인코더 → 양자화 → 디코더 흐름에서, 인코더와 코드북을 **동시에** 업데이트하면:

- 코드북이 계속 변하므로 인코더는 "움직이는 과녁"을 맞춰야 함
- 인코더도 계속 변하므로 코드북은 "달리는 토끼"를 쫓아야 함
- → 학습이 불안정해짐

12.2 해결: 교대 업데이트

Codebook Loss: $\|sg[z_e] - e\|^2$

- $sg[z_e]$: 인코더 출력의 기울기를 **차단** → 코드북만 업데이트
- "인코더는 가만히 있고, 코드북이 인코더 쪽으로 이동"

Commitment Loss: $\|z_e - sg[e]\|^2$

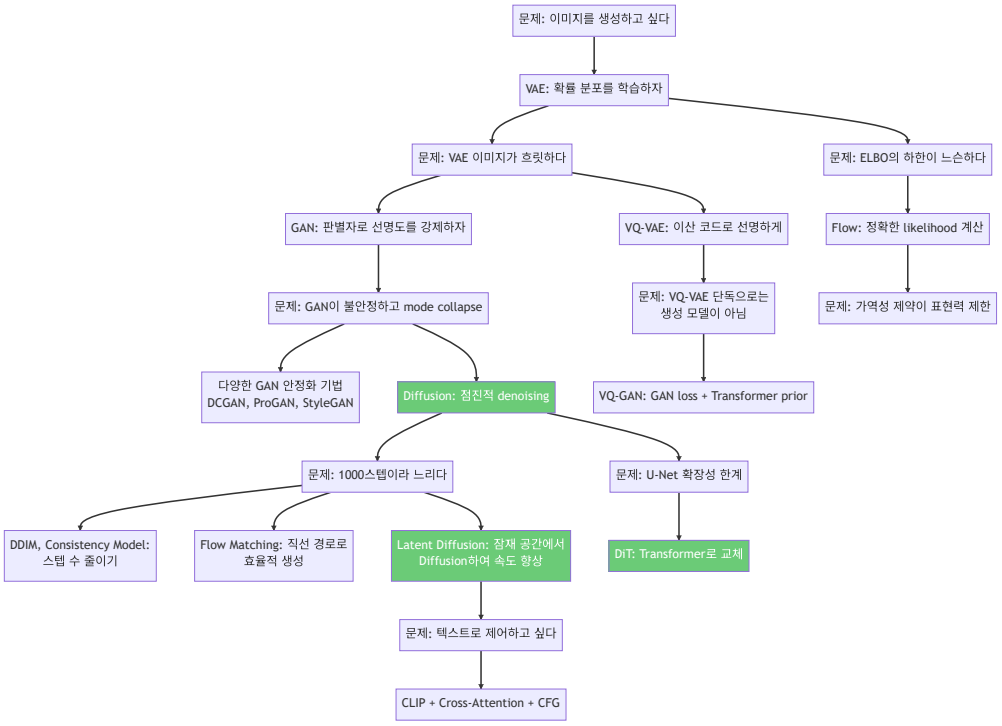
- $sg[e]$: 코드북의 기울기를 **차단** → 인코더만 업데이트
- "코드북은 가만히 있고, 인코더가 코드북 쪽으로 이동"

비유: 두 사람이 동시에 달리며 만나려 하면 어지러운데, 번갈아 한 명씩 이동하면 안정적으로 만날 수 있다.

---

13. 전체 맥락 지도: "문제 → 해결"의 연쇄

아래는 전체 강의를 관통하는 "문제 → 해결 → 새 문제 → 새 해결"의 연쇄다:



핵심 메시지: 어떤 수식도 "갑자기" 나온 것이 아니다. 모든 수식은 **이전 방법의 구체적인 한계를 해결하기 위해** 등장했다. 한계를 먼저 이해하면, 수식이 자연스럽게 따라온다.